

# R Programming for Data Analysis

## DAY 2 — Vectors, Indexing, Lists & Factors

### Day 2 Objective

In Day 1, we learned the fundamentals of R:

**R → RStudio → Variables → Data Types → Operators → Functions → Basic Vectors**

Today we will go one step further and learn how to **store, access, modify, filter, and analyze multiple values**.

By the end of Day 2, students will understand:

- Vectors
- Types of vectors
- Creating vectors
- Indexing
- Vector operations
- Vectorized calculations
- Filtering vectors
- Sorting vectors
- Lists
- Factors
- Practical Data Analyst examples

---

## 1. What is a Vector?

**A vector is a collection of multiple values of the same basic data type.**

For example:

```
sales <- c(10000, 15000, 20000, 18000, 25000)
```

Here, `sales` contains 5 values.

We can think of it as:

| Position | Sales |
|----------|-------|
|----------|-------|

|   |       |
|---|-------|
| 1 | 10000 |
| 2 | 15000 |
| 3 | 20000 |
| 4 | 18000 |
| 5 | 25000 |

## Important

R starts counting positions from **1**, not 0.

---

## 2. Creating a Vector

Use the `c()` function.

`c()` means **combine**.

```
marks <- c(75, 82, 68, 90, 85)
```

Character vector:

```
names <- c("Amit", "Rahul", "Priya", "Neha")
```

Logical vector:

```
result <- c(TRUE, TRUE, FALSE, TRUE)
```

---

## 3. Checking a Vector

View vector

```
marks
```

Check data type

```
class(marks)
```

Check length

```
length(marks)
```

Example:

```
marks <- c(75, 82, 68, 90, 85)
```

```
length(marks)
```

Output:

```
5
```

---

## 4. Accessing Vector Elements — Indexing

Indexing means **accessing a particular value from a vector**.

Syntax:

```
vector_name[position]
```

Example:

```
sales <- c(10000, 15000, 20000, 18000, 25000)
```

First value:

```
sales[1]
```

Output:

```
10000
```

Third value:

```
sales[3]
```

Output:

```
20000
```

Fifth value:

```
sales[5]
```

Output:

```
25000
```

## 5. Selecting Multiple Values

We can select multiple positions.

```
sales[c(1, 3, 5)]
```

Output:

```
10000 20000 25000
```

Select positions 2 and 4:

```
sales[c(2, 4)]
```

## 6. Selecting a Range

We can use `:` to create a sequence.

```
sales[1:3]
```

This means:

```
Position 1 to Position 3
```

Output:

```
10000 15000 20000
```

Another example:

```
sales[2:5]
```

## 7. Excluding Values

Use negative indexing.

```
sales[-1]
```

This removes the first value.

```
sales[-3]
```

This removes the third value.

## Example

```
sales <- c(10000, 15000, 20000, 18000, 25000)

sales[-2]
```

Result:

```
10000 20000 18000 25000
```

---

## 8. Modifying Vector Values

Suppose:

```
sales <- c(10000, 15000, 20000)
```

We want to change the second value.

```
sales[2] <- 18000
```

Now:

```
sales
```

Output:

```
10000 18000 20000
```

---

## 9. Adding Values to a Vector

Use `c()`.

```
sales <- c(10000, 15000, 20000)

sales <- c(sales, 25000)
```

Now:

```
sales
```

Output:

```
10000 15000 20000 25000
```

---

## 10. Removing Values

One simple method is negative indexing.

```
sales <- c(10000, 15000, 20000, 25000)

sales <- sales[-2]
```

The second value is removed.

---

## 11. Vector Arithmetic

R allows us to perform calculations on an entire vector.

Example:

```
sales <- c(10000, 20000, 30000)
```

Add 5000:

```
sales + 5000
```

Output:

```
15000 25000 35000
```

Multiply by 2:

```
sales * 2
```

Output:

```
20000 40000 60000
```

This is called **vectorized operation**.

---

## 12. Why Vectorization is Important for Data Analysts

Suppose we have 5 employees:

```
salary <- c(25000, 30000, 35000, 40000, 45000)
```

If we want to increase every salary by 10%:

```
salary * 1.10
```

Output:

```
27500 33000 38500 44000 49500
```

We don't need a separate calculation for every employee.

This is one of the powerful features of R.

---

## 13. Comparing Vector Values

Suppose:

```
sales <- c(10000, 25000, 18000, 32000, 15000)
```

Check which values are greater than 20000:

```
sales > 20000
```

Output:

```
FALSE TRUE FALSE TRUE FALSE
```

R checks every value individually.

---

## 14. Filtering Values

We can use the logical condition to extract values.

```
sales[sales > 20000]
```

Output:

```
25000 32000
```

This is a very important concept for Data Analysts.

### Meaning

```
sales > 20000
```

creates TRUE/FALSE values.

Then:

```
sales[condition]
```

returns only the values where the condition is `TRUE`.

---

## 15. More Filtering Examples

```
sales <- c(10000, 25000, 18000, 32000, 15000)
```

Sales greater than 15000

```
sales[sales > 15000]
```

Sales less than 20000

```
sales[sales < 20000]
```

Sales equal to 25000

```
sales[sales == 25000]
```

Sales not equal to 15000

```
sales[sales != 15000]
```

---

## 16. Finding Maximum and Minimum

```
sales <- c(10000, 25000, 18000, 32000, 15000)
```

Highest:

```
max(sales)
```

Lowest:

```
min(sales)
```

---

## 17. Sorting a Vector

Ascending order

Use:

```
sort(sales)
```

Example:

```
sales <- c(25000, 10000, 32000, 18000, 15000)

sort(sales)
```

Output:

```
10000 15000 18000 25000 32000
```

## Descending order

```
sort(sales, decreasing = TRUE)
```

Output:

```
32000 25000 18000 15000 10000
```

---

## 18. Sequence Using :

The `:` operator creates a sequence.

```
1:10
```

Output:

```
1 2 3 4 5 6 7 8 9 10
```

Example:

```
5:15
```

---

## 19. Sequence Using `seq()`

For more control, use `seq()`.

```
seq(1, 10)
```

Create values with a gap of 2:

```
seq(2, 20, by = 2)
```

Output:

```
2 4 6 8 10 12 14 16 18 20
```

---

## 20. Repeating Values Using `rep()`

The `rep()` function repeats values.

```
rep(10, times = 5)
```

Output:

```
10 10 10 10 10
```

Example:

```
rep(c("Sales", "HR"), times = 3)
```

Output:

```
"Sales" "HR" "Sales" "HR" "Sales" "HR"
```

---

## 21. Lists in R

A **list** can contain different types of data.

Unlike a vector, a list can contain:

- Character
- Numeric
- Logical
- Vector
- Another list

Example:

```
employee <- list(  
  name = "Amit",  
  age = 28,  
  salary = 45000,
```

```
skills = c("Excel", "SQL", "Power BI")
)
```

View the list:

```
employee
```

---

## 22. Accessing List Elements

Access by name:

```
employee$name
```

Output:

```
"Amit"
```

Salary:

```
employee$salary
```

Skills:

```
employee$skills
```

First skill:

```
employee$skills[1]
```

Output:

```
"Excel"
```

---

## 23. Vector vs List

| Feature                | Vector | List |
|------------------------|--------|------|
| Stores multiple values | Yes    | Yes  |

|                                  |              |                      |
|----------------------------------|--------------|----------------------|
| Same basic type                  | Usually yes  | No                   |
| Can contain different data types | No           | Yes                  |
| Example                          | Sales values | Employee information |

## Vector

```
sales <- c(10000, 20000, 30000)
```

## List

```
employee <- list(
  name = "Amit",
  age = 28,
  salary = 45000
)
```

## 24. Factors in R

A **factor** is used to represent categorical data.

Categorical data means values that belong to categories.

Examples:

```
Gender
Department
City
Product Category
Education
```

Example:

```
department <- factor(c(
  "Sales",
  "HR",
  "IT",
  "Sales",
  "IT"
))
```

View:

```
department
```

## 25. Levels in a Factor

Use:

```
levels(department)
```

This displays the unique categories.

Example output:

```
"HR" "IT" "Sales"
```

Number of categories:

```
nlevels(department)
```

## 26. Why Factors Matter in Data Analysis

Suppose an employee dataset contains:

```
Sales
HR
IT
Sales
Marketing
IT
```

These are not continuous numerical values.

They are **categories**.

R can represent them as factors:

```
department <- factor(c(  
  "Sales",  
  "HR",  
  "IT",  
  "Sales",  
  "Marketing",  
  "IT"  
))
```

Factors are particularly useful in statistical analysis and categorical data handling.

---

## 27. Real Data Analyst Example

Suppose we have employee salaries:

```
salary <- c(  
  25000,  
  32000,  
  28000,  
  45000,  
  50000,  
  38000  
)
```

Average salary

```
mean(salary)
```

Highest salary

```
max(salary)
```

Employees earning more than ₹35,000

```
salary[salary > 35000]
```

Sort salary from highest to lowest

```
sort(salary, decreasing = TRUE)
```

Give everyone a 10% increment

```
salary * 1.10
```

This is a basic example of how R can be used for **HR Data Analysis**.

---

## Important Day 2 Functions

| Function<br>/<br>Operator | Purpose                     |
|---------------------------|-----------------------------|
| <code>c()</code>          | Combine values              |
| <code>[]</code>           | Access/index values         |
| <code>\$</code>           | Access list element by name |
| <code>:</code>            | Create sequence             |
| <code>seq()</code>        | Create controlled sequence  |
| <code>rep()</code>        | Repeat values               |
| <code>sort()</code>       | Sort values                 |

|                        |                           |
|------------------------|---------------------------|
| <code>max()</code>     | Highest value             |
| <code>min()</code>     | Lowest value              |
| <code>length()</code>  | Number of values          |
| <code>factor()</code>  | Create categorical data   |
| <code>levels()</code>  | Display factor categories |
| <code>nlevels()</code> | Count categories          |



## DAY 2 PRACTICE QUESTIONS

### Practice 1 — Vector Creation

Create a vector named `marks` :

```
65, 78, 82, 55, 90, 72, 88
```

Find:

1. Number of students
  2. Highest marks
  3. Lowest marks
  4. Average marks
  5. Total marks
- 

### Practice 2 — Indexing

Using:

```
marks <- c(65, 78, 82, 55, 90, 72, 88)
```

Find:

1. First mark
  2. Third mark
  3. Last mark
  4. Second and fifth marks
  5. First three marks
- 

## Practice 3 — Filtering

Using the same `marks` vector:

Find students who scored:

1. More than 70
  2. Less than 60
  3. Equal to 90
  4. Greater than or equal to 80
- 

## Practice 4 — Salary Analysis

Create:

```
salary <- c(25000, 32000, 28000, 45000, 50000, 38000)
```

Perform:

1. Sort ascending
  2. Sort descending
  3. Find average salary
  4. Find highest salary
  5. Find employees earning above ₹35,000
  6. Calculate salary after a 10% increment
- 

## Practice 5 — Department Factor

Create:

```
department <- factor(c(  
  "Sales",  
  "HR",  
  "IT",  
  "Sales",  
  "Marketing",  
  "IT",  
  "HR"  
))
```

Find:

1. All departments
2. Unique levels
3. Number of levels

---

## ★ DAY 2 ASSIGNMENT — Employee Analysis

Create the following vectors:

```
employee <- c(  
  "Amit",  
  "Rahul",  
  "Priya",  
  "Neha",  
  "Karan"  
)  
  
salary <- c(  
  30000,  
  45000,  
  38000,  
  52000,  
  35000  
)  
  
department <- factor(c(  
  "Sales"
```

### Questions

1. Display all employee names.
2. Display the third employee.

3. Display all salaries.
4. Find the average salary.
5. Find the highest salary.
6. Find the lowest salary.
7. Find employees earning more than ₹40,000.
8. Sort salaries from highest to lowest.
9. Calculate salary after a 10% increment.
10. Display the department categories.